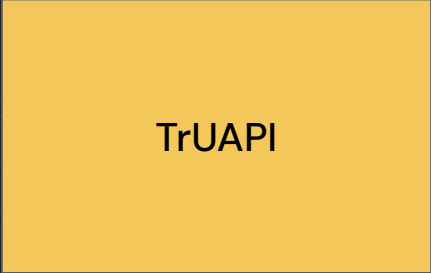


Behind the Polkadot Stack

Torsten Stüber | Parity

PBA Lisbon, April 2026

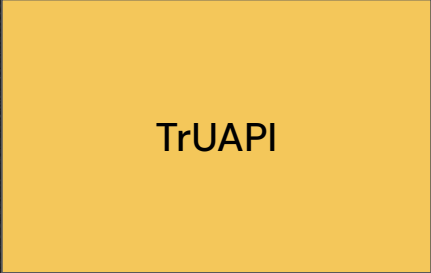
What we'll cover



TrUAPI

A sandboxed
product's connection
to the world

What we'll cover



TrUAPI

A sandboxed
product's connection
to the world



Bulletin Chain
Statement Store

Two data layers

Advanced Substrate
features

What we'll cover

TrUAPI

A sandboxed
product's connection
to the world

Bulletin Chain
Statement Store

Two data layers

Advanced Substrate
features

Ring VRF

Cryptographic
primitive

Used for anonymous
but accountable
action

What we'll cover

TrUAPI

A sandboxed
product's connection
to the world

Bulletin Chain
Statement Store

Two data layers

Advanced Substrate
features

Ring VRF

Cryptographic
primitive

Used for anonymous
but accountable
action

Ring VRF after a break.

Section 1

Triangle hosts & TrUAPI

The tension



My keys,
my funds,
my identity.

I trust this.

The tension



My keys,
my funds,
my identity.

I trust this.



But who wrote
all of these

The tension



My keys,
my funds,
my identity.

I trust this.



But who wrote
all of these

This is the problem

How do you lock this down?

How to run arbitrary third-party code in wallet
without putting wallet at risk?

How do you lock this down?

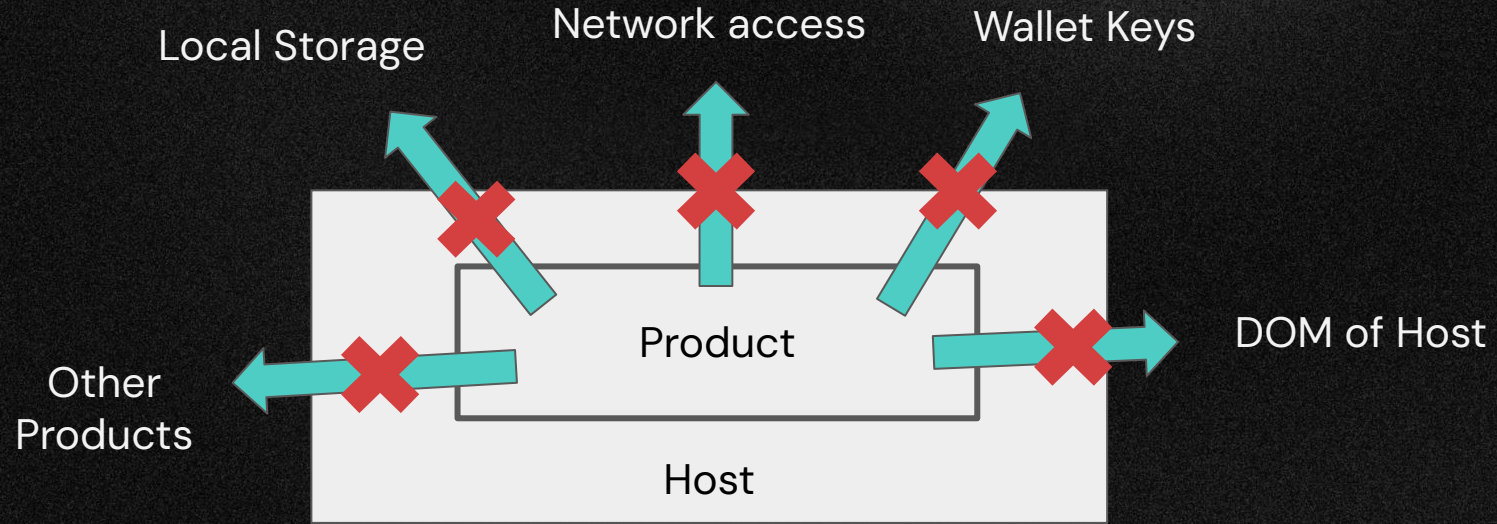
How to run arbitrary third-party code in wallet
without putting wallet at risk?

Answer: Sandbox

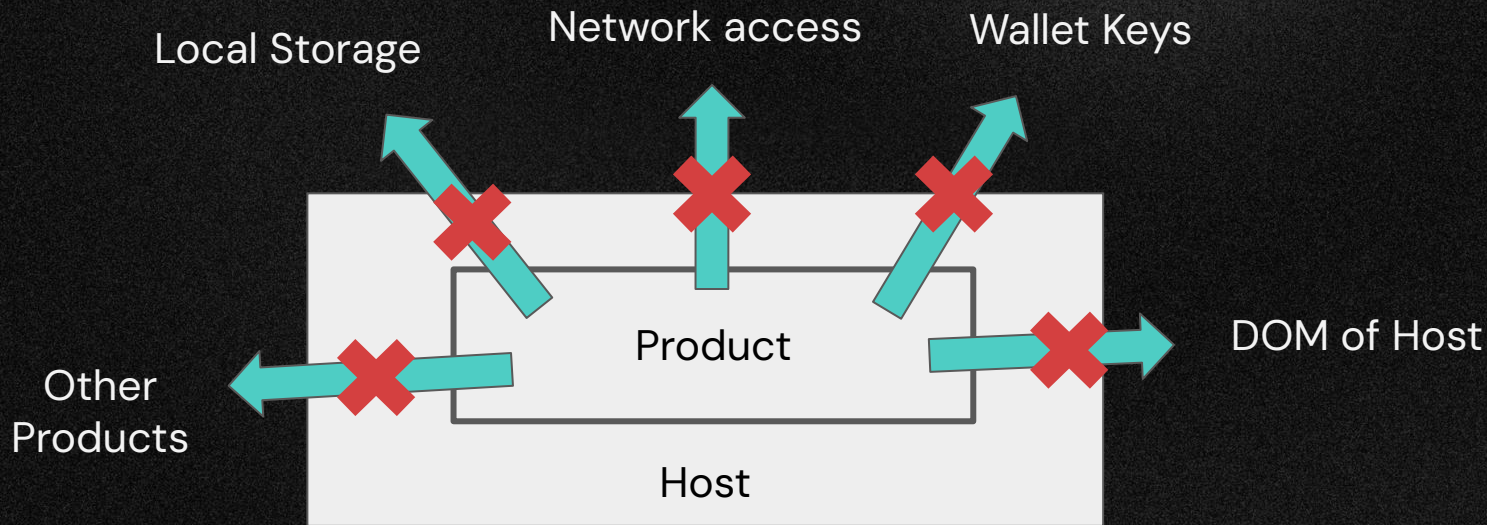
The sandbox, concretely



The sandbox, concretely

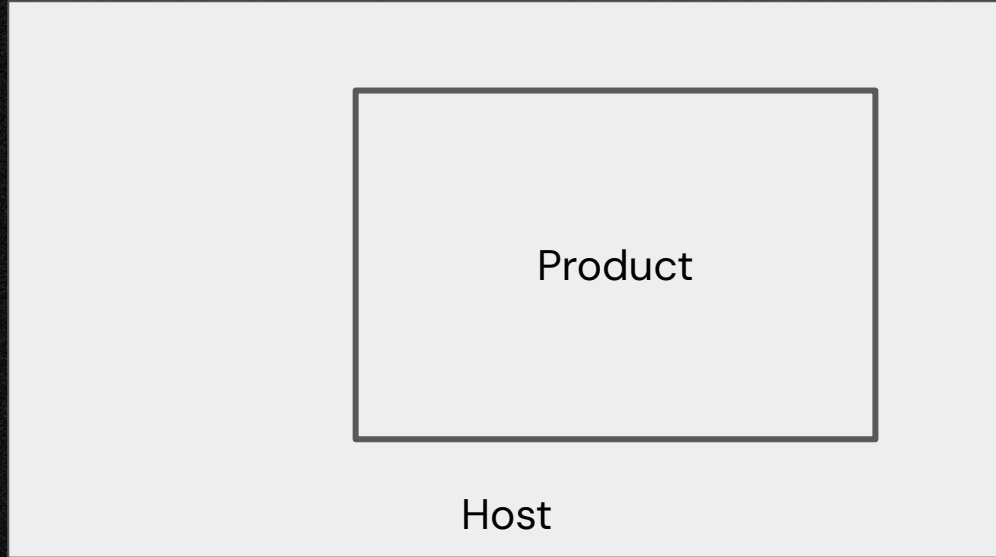


The sandbox, concretely

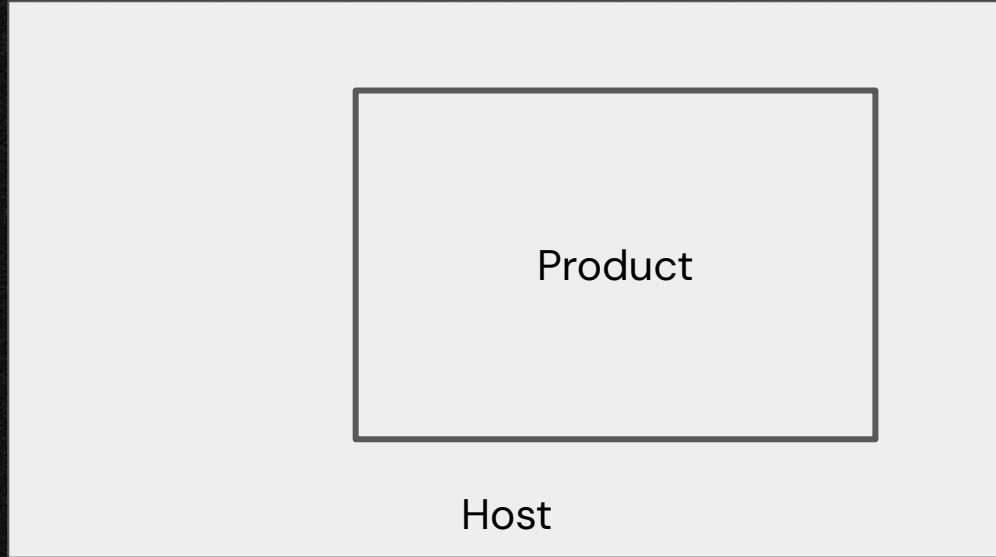


Iframe + strict CSP on web
WebView on mobile/desktop

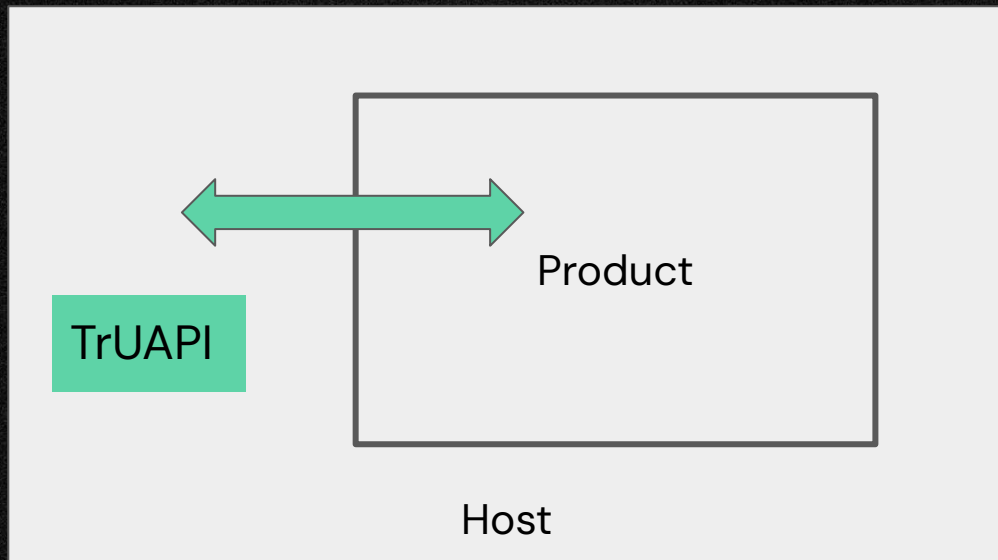
The one door out



The one door out



The one door out



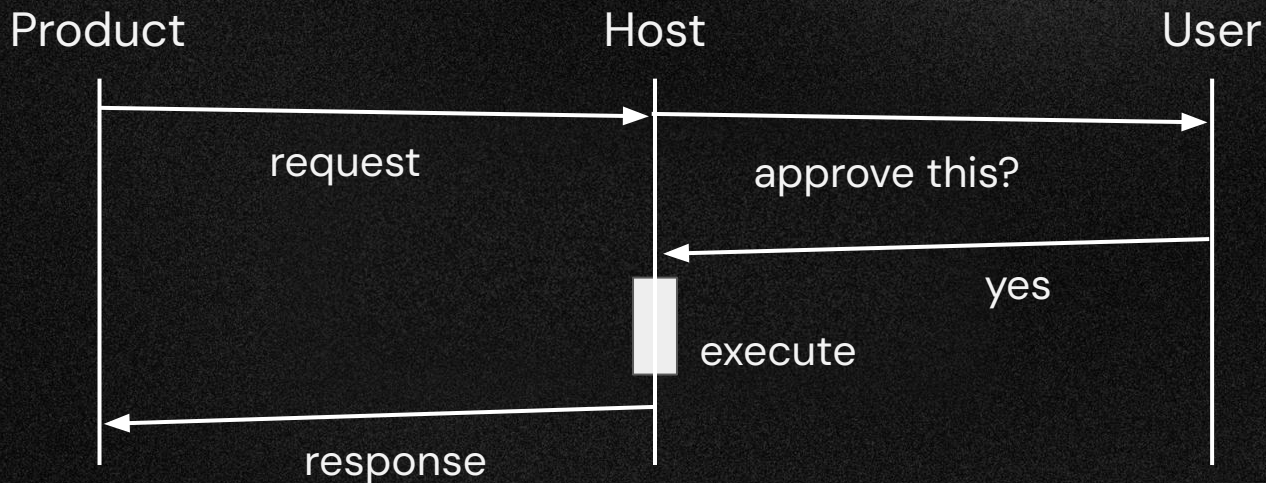
TrUAPI

- `postMessage` (web)
- Native bridge (mobile)

What the wire carries

- Host Calls: *feature detection, navigation, push notifications*
- Permissions: *camera, microphone, Bluetooth, remote endpoint approvals*
- Local Storage: *key-value storage per product*
- Account Management: *derived accounts, aliases, Ring VRF proofs*
- Signing: *payload signing, raw signing, transaction creation*
- Chain Interaction: *full Substrate JSON-RPC over the wire*
- Chat: *room/bot management, messaging, custom message rendering*
- Statement Store: *subscribe, prove, submit cryptographic statements*
- Preimage: *lookup and submit preimages*

How a call works



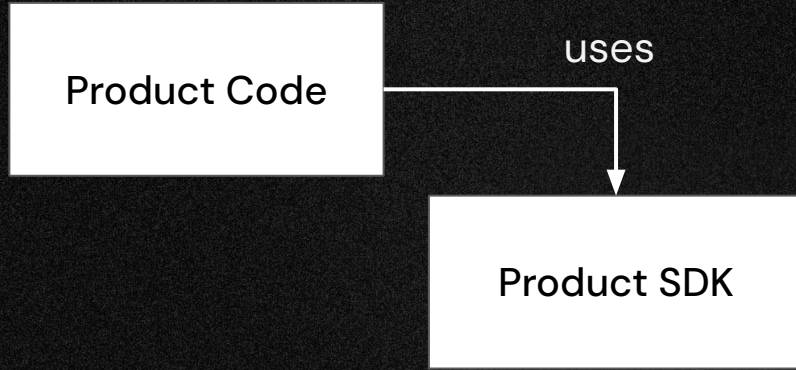
Patterns:

- Request-Response
- Subscription (host pushes updates)

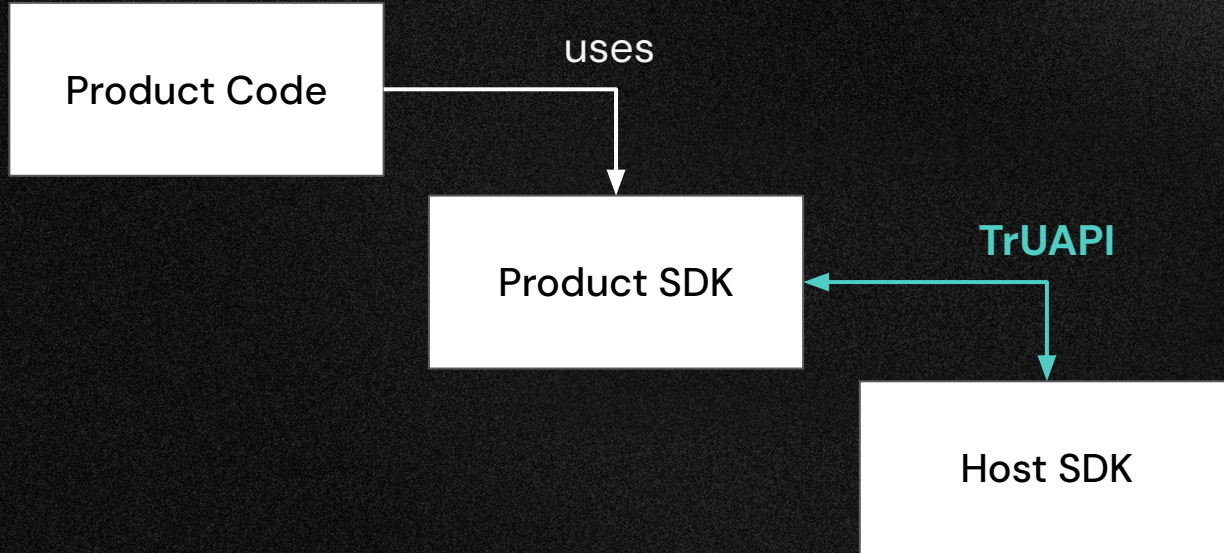
Two libraries wrap this

Product Code

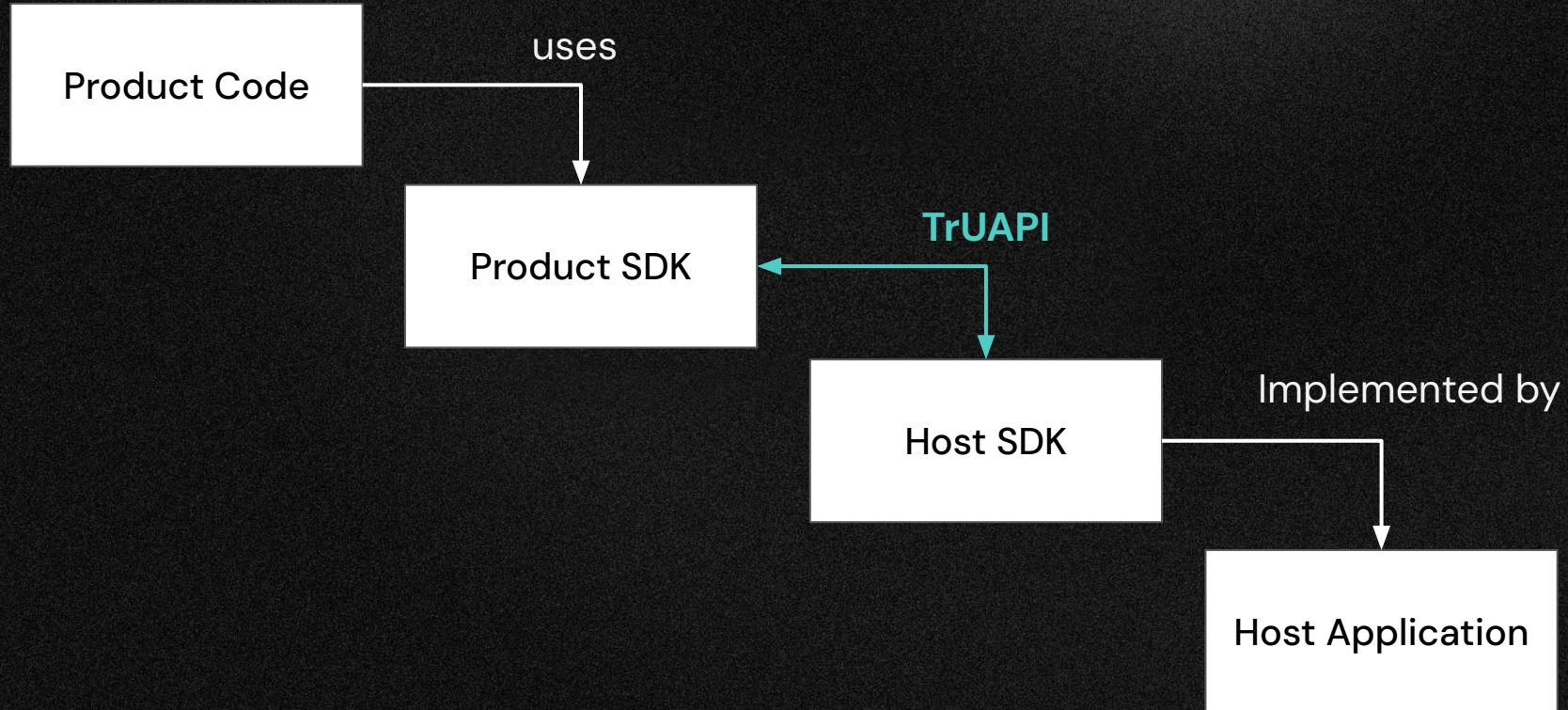
Two libraries wrap this



Two libraries wrap this



Two libraries wrap this



Let's see the wire

Debug view, live.

What you just saw

- Product asked for an account
 - It got a public key
 - No private key crossed the wire

What you just saw

- Product asked for an account
 - It got a public key
 - No private key crossed the wire
- Product queried the chain
 - Host talks to a light client (Smoldot)
 - Product just sees JSON-RPC response

What you just saw

- Product asked for an account
 - It got a public key
 - No private key crossed the wire
- Product queried the chain
 - Host talks to a light client (Smoldot)
 - Product just sees JSON-RPC response
- Product asked Host to sign
 - User approved
 - Signed payload came back through wire

Systems hiding behind the wire



Statement Store

- Chat messages
- Communication with Polkadot Mobile

Systems hiding behind the wire



Statement Store

- Chat messages
- Communication with Polkadot Mobile



Bulletin Chain

- Product SPA
- File storage

Systems hiding behind the wire



Statement Store

- Chat messages
- Communication with Polkadot Mobile



Bulletin Chain

- Product SPA
- File storage



Ring VRF

- Cryptographic Primitive
- For accounts and alisases

Section 2

Bulletin Chain

The question

How to

- store 2 MB files on-chain,
- keep blocks small, **and**
- force every block author to prove they have the data?

And do all of it with Substrate features.

Four Substrate moments

1. Data lives outside the state trie

Four Substrate moments

1. Data lives outside the state trie
2. A mandatory inherent proves retention every block

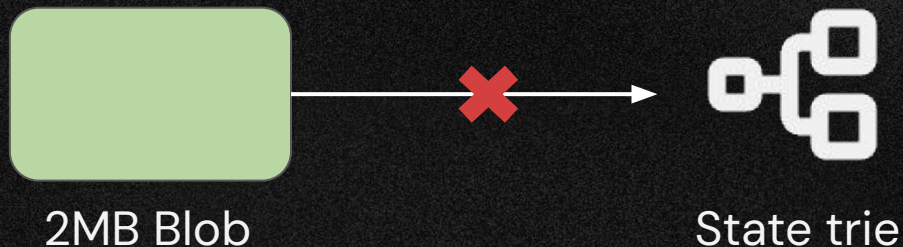
Four Substrate moments

1. Data lives outside the state trie
2. A mandatory inherent proves retention every block
3. A transaction extension transforms the signer's origin

Four Substrate moments

1. Data lives outside the state trie
2. A mandatory inherent proves retention every block
3. A transaction extension transforms the signer's origin
4. Feeless writes, authorized only by another parachain

The problem: bulk data on-chain



- State sync downloads it
- State root hashes it
- Light-client proofs traverse past it
- Block execution touches it on every write

Two storage worlds

State Trie

- Consensus-critical
- All nodes sync it
- Part of the state root
- Runtime can read/write

Two storage worlds

State Trie

- Consensus-critical
- All nodes sync it
- Part of the state root
- Runtime can read/write

TRANSACTION column

- Node-local DB
- Not in the state root
- Not consensus-critical
- Runtime cannot read it

Two storage worlds

State Trie

- Consensus-critical
- All nodes sync it
- Part of the state root
- Runtime can read/write

TRANSACTION column

- Node-local DB
- Not in the state root
- Not consensus-critical
- Runtime cannot read it



A host-function trick

```
pallet-transaction-storage::store(data)
```

- `StorageMap::insert(`
 `content_hash,`
 `metadata`
 `)`

A host-function trick

```
pallet-transaction-storage::store(data)
```

- `StorageMap::insert(`
 `content_hash,`
 `metadata`
 `)`
- `sp_io::transaction_index::index(`
 `extrinsic_idx,`
 `data_len,`
 `content_hash`
 `)`

A host-function trick

```
pallet-transaction-storage::store(data)
```

- `StorageMap::insert(`
 `content_hash,`
 `metadata`
`)`
- `sp_io::transaction_index::index(`
 `extrinsic_idx,`
 `data_len,`
 `content_hash`
`)`

- Doesn't write immediately
- emits `IndexOperation` entry into the overlay changeset
- When block commits, node processes queued operations
 - Extract blob from the extrinsic
 - Store in `TRANSACTION` column
- Rollbacks discard
- Dry-runs never execute them

Why would a node actually store the data?

TRANSACTION column lives outside consensus.
Why would any node bother to store it?

Why would a node actually store the data?

TRANSACTION column lives outside consensus.
Why would any node bother to store it?

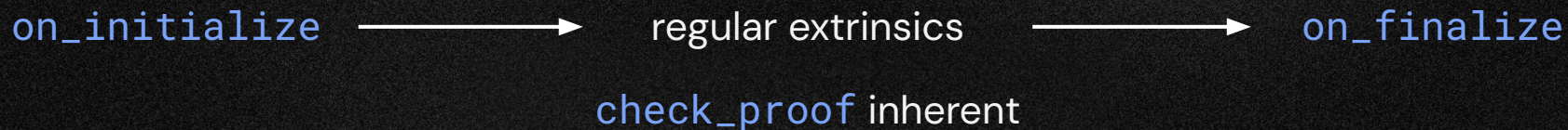
Runtime can't directly check what's on the node's disk.

How to enforce storage without relying on good behaviour?

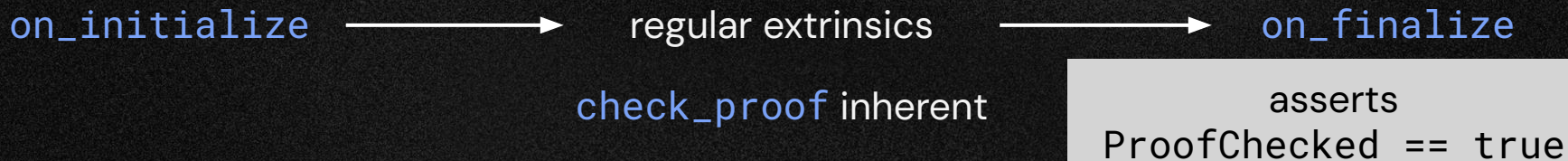
The mandatory inherent

`on_initialize` → regular extrinsics → `on_finalize`

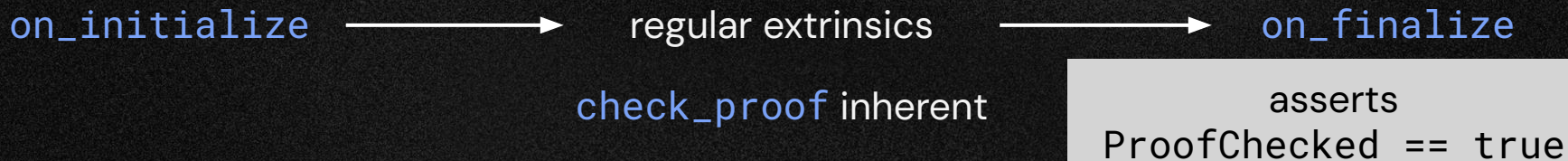
The mandatory inherent



The mandatory inherent



The mandatory inherent



Every block, the author must include this inherent and pass its check.
If they can't, they can't produce the block at all.

What the inherent proves

- Current block **N**
 - Target: Prove you still have data from block **N – RetentionPeriod**

What the inherent proves

- Current block **N**
 - Target: Prove you still have data from block **N - RetentionPeriod**
 - Target block had **K** stored files
- Each has *chunk trie root* in on-chain metadata.

What the inherent proves

- Current block **N**
 - Target: Prove you still have data from block **N - RetentionPeriod**
- Target block had **K** stored files
Each has *chunk trie root* in on-chain metadata.
- Pick random chunk, using `parent_hash` as the seed:
 - `chunk_index = hash(parent_hash) % total_chunks`
 - Select one specific chunk of one specific file

What the inherent proves

- Current block **N**
 - Target: Prove you still have data from block **N - RetentionPeriod**
- Target block had **K** stored files
Each has *chunk trie root* in on-chain metadata.
- Pick random chunk, using `parent_hash` as the seed:
 - `chunk_index = hash(parent_hash) % total_chunks`
 - Select one specific chunk of one specific file
- Author reads full blob from their TRANSACTION column
 - re-chunk, build Merkle-trie proof of selected chunk, pass as the inherent's argument

What the inherent proves

- Current block **N**
 - Target: Prove you still have data from block **N - RetentionPeriod**
- Target block had **K** stored files
Each has *chunk trie root* in on-chain metadata.
- Pick random chunk, using `parent_hash` as the seed:
 - `chunk_index = hash(parent_hash) % total_chunks`
 - Select one specific chunk of one specific file
- Author reads full blob from their TRANSACTION column
 - re-chunk, build Merkle-trie proof of selected chunk, pass as the inherent's argument
- Runtime verifies proof against chunk trie root stored on-chain

What the inherent proves

- Current block **N**
 - Target: Prove you still have data from block **N - RetentionPeriod**
- Target block had **K** stored files
Each has *chunk trie root* in on-chain metadata.
- Pick random chunk, using `parent_hash` as the seed:
 - `chunk_index = hash(parent_hash) % total_chunks`
 - Select one specific chunk of one specific file
- Author reads full blob from their TRANSACTION column
 - re-chunk, build Merkle-trie proof of selected chunk, pass as the inherent's argument
- Runtime verifies proof against chunk trie root stored on-chain

If author lost data, they can't produce proof

The authorization system

Storage Map `Authorizations`

- Key: `AuthorizationScope`
 - `Account(who)` or
 - `Preimage(hash)`
- Value
 - `transactions_remaining,`
 - `bytes_remaining,`
 - `expiration`

The authorization system

Storage Map `Authorizations`

- Key: `AuthorizationScope`
 - `Account(who)` or
 - `Preimage(hash)`
- Value
 - `transactions_remaining,`
 - `bytes_remaining,`
 - `expiration`

The flow:

- An authorizer pre-allocates quota to an account
- Authorized account submits store or renew
- Pallet decrements the quota
- Once either counter hits zero, authorization is deleted.

The authorization system

Storage Map Authorizations

- Key: `AuthorizationScope`
 - `Account(who)` or
 - `Preimage(hash)`
- Value
 - `transactions_remaining,`
 - `bytes_remaining,`
 - `expiration`

The flow:

- An authorizer pre-allocates quota to an account
- Authorized account submits store or renew
- Pallet decrements the quota
- Once either counter hits zero, authorization is deleted.

No fees. The quota is the spam-control mechanism

The check lives in a transaction extension

The TxExtension pipeline

`validate()` $\longrightarrow$ `prepare()` $\longrightarrow$ `dispatch()` $\longrightarrow$ `post_dispatch()`

The check lives in a transaction extension

The TxExtension pipeline

`validate()` → `prepare()` → `dispatch()` → `post_dispatch()`

Bulletin's extension `ValidateStorageCalls`

The check lives in a transaction extension

The `TxExtension` pipeline

`validate()` → `prepare()` → `dispatch()` → `post_dispatch()`

Bulletin's extension `ValidateStorageCalls`

`validate()` runs during tx-pool inclusion

- Checks the signer has valid authorization for this call
- If yes, transforms the origin (next slide)
- If no, rejects the transaction from the pool

Invalid transactions die at the tx-pool stage

The check lives in a transaction extension

The `TxExtension` pipeline

`validate()` $\longrightarrow$ `prepare()` $\longrightarrow$ `dispatch()` $\longrightarrow$ `post_dispatch()`

Bulletin's extension `ValidateStorageCalls`

`validate()` runs during tx-pool inclusion

- Checks the signer has valid authorization for this call
- If yes, transforms the origin (next slide)
- If no, rejects the transaction from the pool

Invalid transactions die at the tx-pool stage

`prepare()` runs just before dispatch

- Consumes the authorization: decrements the quota
- If counter hits zero, deletes the authorization entirely

The origin transformation

Before `ValidateStorageCalls::validate()`

Origin: `Origin::Signed(alice)`

The origin transformation

Before `ValidateStorageCalls::validate()`

Origin: `Origin::Signed(alice)`

After `ValidateStorageCalls::validate()`

Origin: `Origin::Authorized { who: alice, scope: Account(alice) }`

The origin transformation

Before `ValidateStorageCalls::validate()`

Origin: `Origin::Signed(alice)`

After `ValidateStorageCalls::validate()`

Origin: `Origin::Authorized { who: alice, scope: Account(alice) }`

In the pallet body

```
pub fn store(origin: OriginFor<T>, data: Vec<u8>) -> DispatchResult {  
    let (who, scope) = ensure_authorized:<T>(origin)?; // ← this  
    // ...  
}
```


No fees. At all.

The `store` extrinsic

```
#[pallet::call_index(0)]  
#[pallet::feeless_if(|_, _, _| true)]  
pub fn store(origin: OriginFor<T>, data: Vec<u8>) -> DispatchResult {  
    ...  
}
```


No fees. At all.

The `store` extrinsic

```
#[pallet::call_index(0)]  
#[pallet::feeless_if(|_, _, _| true)]  
pub fn store(origin: OriginFor<T>, data: Vec<u8>) -> DispatchResult {  
    ...  
}
```

The chain's `Currency` type

```
type Currency = NoCurrency; // all balances are zero
```


No fees. At all.

The `store` extrinsic

```
#[pallet::call_index(0)]
#[pallet::feeless_if(|_, _, _| true)]
pub fn store(origin: OriginFor<T>, data: Vec<u8>) -> DispatchResult {
    ...
}
```

The chain's `Currency` type

```
type Currency = NoCurrency; // all balances are zero
```

Bulletin Chain has no fee token. Storage is free to users.
The authorization system is the only spam control.

Who can authorize?

The Authorizer origin on production Polkadot Bulletin

```
type Authorizer = EitherOfDiverse<  
  EnsureRoot,  
  EnsureSignedBy<TestAccounts>,  
>;
```


Who can authorize?

The Authorizer origin on production Polkadot Bulletin

```
type Authorizer = EitherOfDiverse<  
  EnsureRoot,  
  EnsureSignedBy<TestAccounts>,  
>;
```

The XCM config

```
LocationAsSuperuser<Equals<PeoplePolkadotLocation>>
```

When the People chain sends an XCM
Transact, the call runs as Root.

Who can authorize?

The Authorizer origin on production Polkadot Bulletin

```
type Authorizer = EitherOfDiverse<  
  EnsureRoot,  
  EnsureSignedBy<TestAccounts>,  
>;
```

The XCM config

```
LocationAsSuperuser<Equals<PeoplePolkadotLocation>>
```

When the People chain sends an XCM **Transact**, the call runs as Root.

Only the People chain can authorize storage, via XCM

The lesson

Block authors who lose the data cannot author the next block.

The lesson

Block authors who lose the data cannot author the next block.

1. TRANSACTION column
bulk data outside consensus
2. Mandatory inherent
proof-of-retention every block
3. Transaction extension
origin transformation, spam rejected early
4. Feeless + XCM-only
authorization delegated to the People chain.

Next: a completely different design



Bulletin Chain

- Bulk content
- On-chain metadata
- 14-day retention enforced by liveness

Next: a completely different design



Bulletin Chain

- Bulk content
- On-chain metadata
- 14-day retention enforced by liveness



Statement Store

- Ephemeral signed messages
- Lightweight
- Gossiped peer-to-peer
- Not on-chain at all

Section 3

Statement Store

What is a statement?

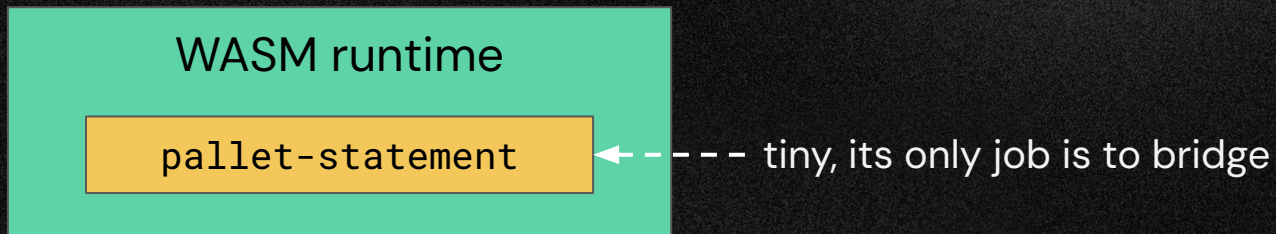
```
Statement {  
    authenticity_proof: Proof,           // Sr25519 / Ed25519 / Secp256k1 / OnChain  
    expiry: u64,                         // when to drop  
    channel: Option<[u8; 32]>,           // 1 message per channel per account  
    topics: [Option<[u8; 32]>; 4],       // up to 4 indexable tags  
    decryption_key: Option<[u8; 32]>,    // enables ECIES encryption  
    data: Vec<u8>,                      // payload  
}
```


What is a statement?

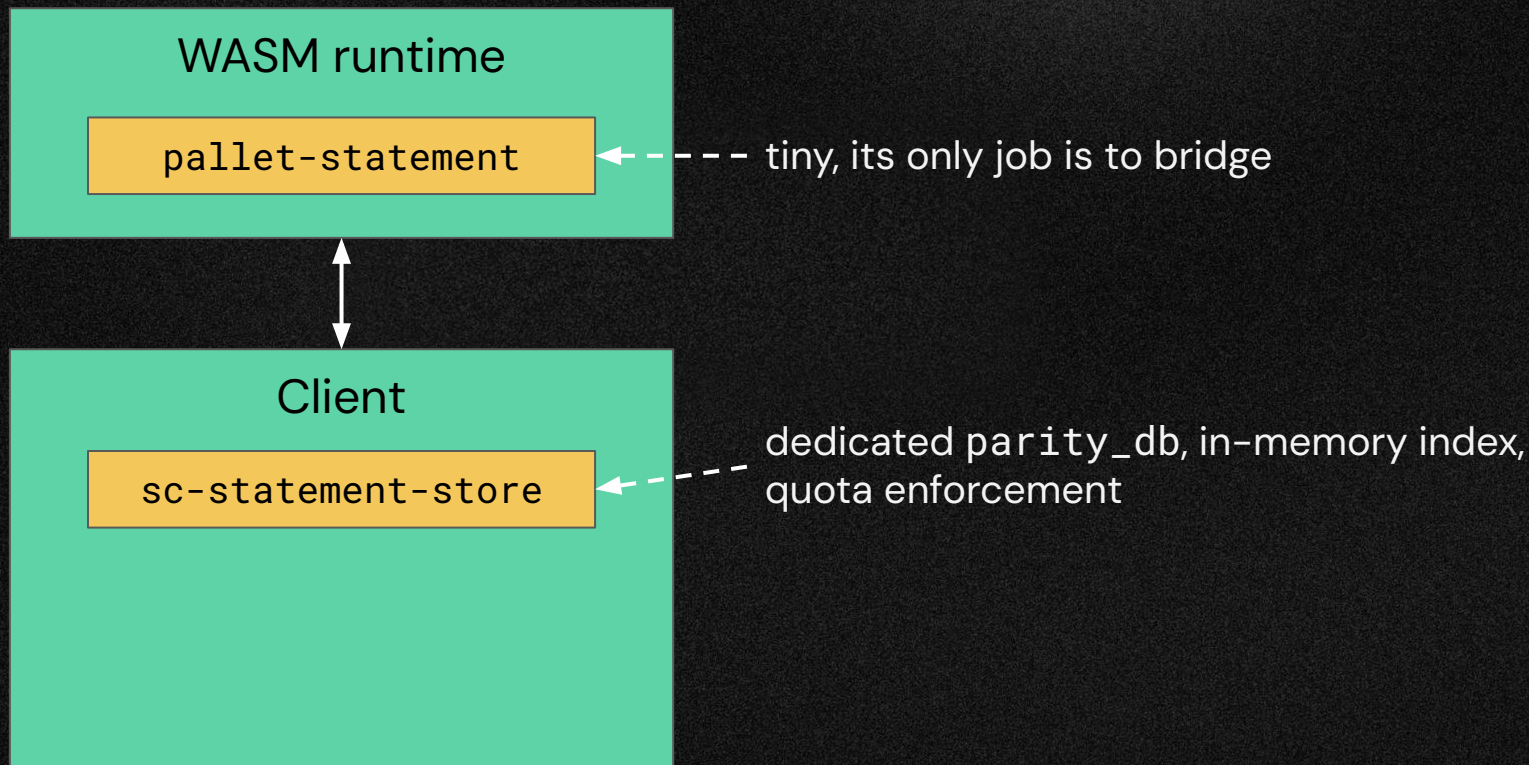
```
Statement {  
    authenticity_proof: Proof,           // Sr25519 / Ed25519 / Secp256k1 / OnChain  
    expiry: u64,                         // when to drop  
    channel: Option<[u8; 32]>,           // 1 message per channel per account  
    topics: [Option<[u8; 32]>; 4],       // up to 4 indexable tags  
    decryption_key: Option<[u8; 32]>,   // enables ECIES encryption  
    data: Vec<u8>,                      // payload  
}
```

- Signed SCALE bundle
- Up to 4 topics for filtering
- Optional per-recipient encryption
- Self-expiring: clients drop after expiry
- Max ~1 MB encoded

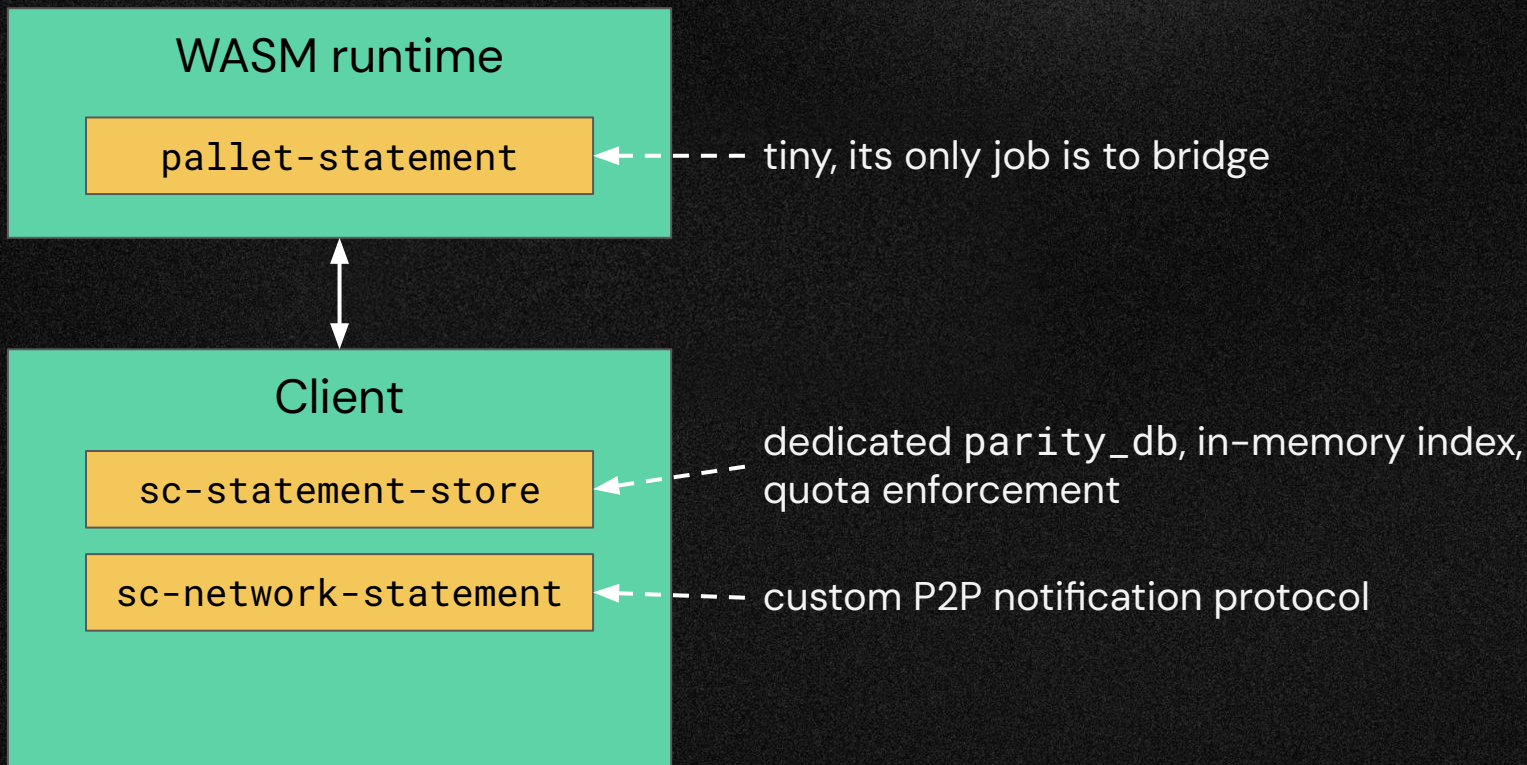
A fully custom subsystem



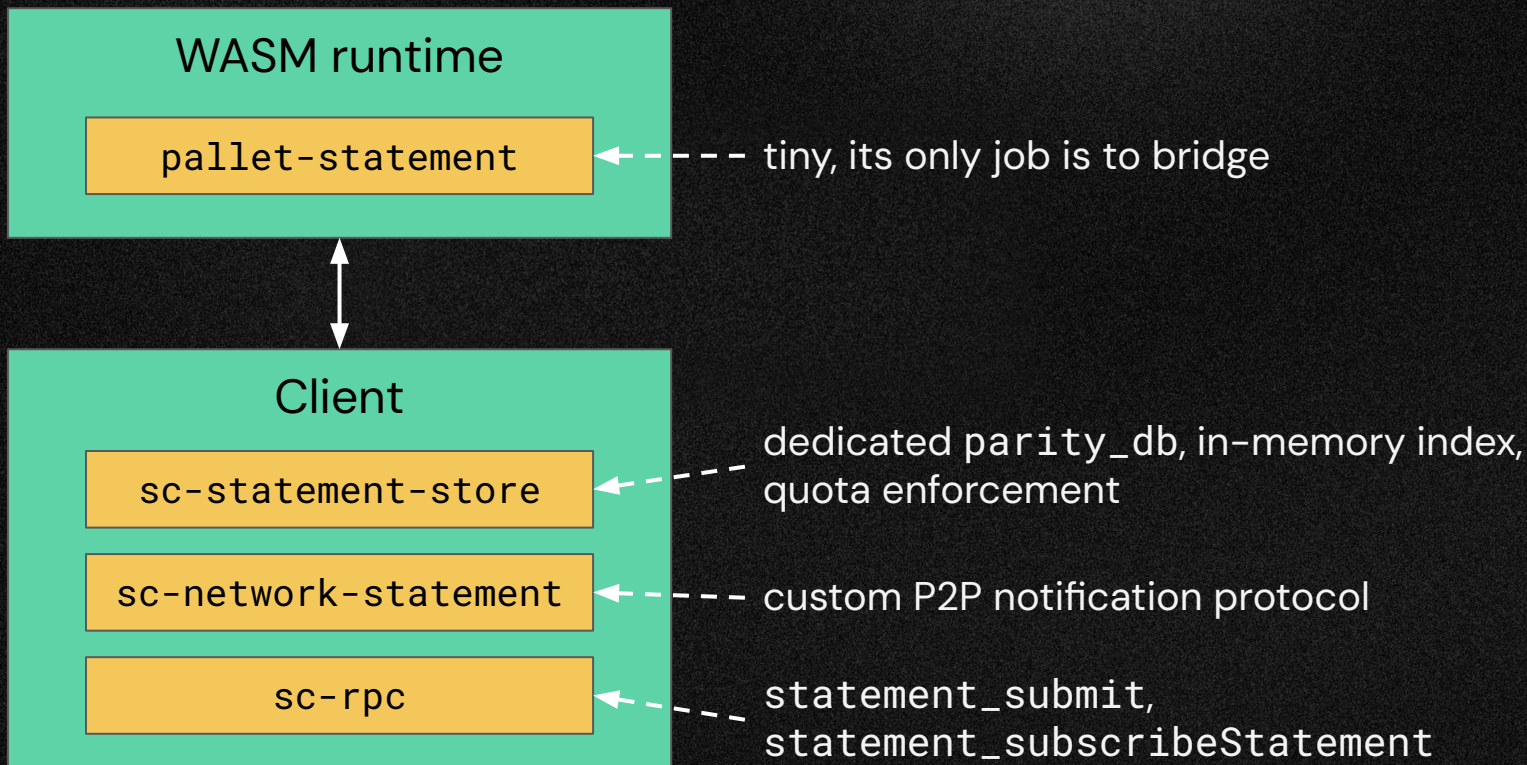
A fully custom subsystem



A fully custom subsystem



A fully custom subsystem



Offchain workers as the bridge

- Block **N** imported (becomes new best block)

Offchain workers as the bridge

- Block **N** imported (becomes new best block)
- `pallet-statement::offchain_worker(N)` runs

Offchain workers as the bridge

- Block **N** imported (becomes new best block)
- `pallet-statement::offchain_worker(N)` runs
- Reads `NewStatement` events from block **N**

Offchain workers as the bridge

- Block **N** imported (becomes new best block)
- `pallet-statement::offchain_worker(N)` runs
- Reads `NewStatement` events from block **N**
- For each event, wraps the statement with
`OnChain { block_hash: N, event_index }`

Offchain workers as the bridge

- Block **N** imported (becomes new best block)
- `pallet-statement::offchain_worker(N)` runs
- Reads `NewStatement` events from block **N**
- For each event, wraps the statement with
`OnChain { block_hash: N, event_index }`
- Calls `sp_statement_store::submit_statement` (host function)

Offchain workers as the bridge

- Block **N** imported (becomes new best block)
- `pallet-statement::offchain_worker(N)` runs
- Reads `NewStatement` events from block **N**
- For each event, wraps the statement with
`OnChain { block_hash: N, event_index }`
- Calls `sp_statement_store::submit_statement` (host function)
- Statement lands in `sc-statement-store`
gets indexed, gets gossiped to peers

Offchain workers as the bridge

- Block **N** imported (becomes new best block)
- `pallet-statement::offchain_worker(N)` runs
- Reads `NewStatement` events from block **N**
- For each event, wraps the statement with
`OnChain { block_hash: N, event_index }`
- Calls `sp_statement_store::submit_statement` (host function)
- Statement lands in `sc-statement-store`
gets indexed, gets gossiped to peers

On-chain → off-chain
The unusual direction

Per-account quotas

```
#[pallet::storage]
pub type Allowance<T> =
    StorageMap<_,
        Blake2_128Concat,
        AccountId,
        StatementAllowance
    >;
```

Normal pallet storage. Prefix hashed.
Key hashed.

Per-account quotas

```
#[pallet::storage]
pub type Allowance<T> =
    StorageMap<_,
        Blake2_128Concat,
        AccountId,
        StatementAllowance
    >;
```

Normal pallet storage. Prefix hashed.
Key hashed.

sc-statement-store

- Key:
`b":statement_allowance:" ||
account_id`
- Value:
`StatementAllowance { max_count,
max_size }`

Raw unhashed storage at a fixed prefix
client reads this directly during submission

Per-account quotas

```
#[pallet::storage]
pub type Allowance<T> =
    StorageMap<_,
        Blake2_128Concat,
        AccountId,
        StatementAllowance
    >;
```

Normal pallet storage. Prefix hashed.
Key hashed.

sc-statement-store

- Key:
`b":statement_allowance:" || account_id`
- Value:
`StatementAllowance { max_count, max_size }`

Raw unhashed storage at a fixed prefix

client reads this directly during submission

Quota source: on-chain. Enforcement: off-chain, in the client

The lesson

	Bulletin Chain	Statement Store
What it stores	Bulk content, blobs	Signed messages
Persistence	~14 days, proof-of-retention	Ephemeral, self-expiring
Consensus involvement	On-chain metadata + TRANSACTION column	None, off-chain entirely

Section 4

Ring VRF

Back to the cryptography

- Remember the People Chain? The Ring VRF extension in the Host SDK

Back to the cryptography

- Remember the People Chain? The Ring VRF extension in the Host SDK
- Ring VRF: the same primitive enables many different forms of anonymous-but-accountable action
 - Across any chain in Polkadot.

Back to the cryptography

- Remember the People Chain? The Ring VRF extension in the Host SDK
- Ring VRF: the same primitive enables many different forms of anonymous-but-accountable action
 - Across any chain in Polkadot.
- We'll build the primitive from first principles
 - No crypto prerequisites

Ring VRF, in context



People Chain



Another parachain

Ring roots propagate via XCM (`members-notifier` → `members-subscriber`)

Ring VRF, in context



People Chain



Another parachain

Ring roots propagate via XCM (`members-notifier` → `members-subscriber`)

you saw on Bulletin:

- Custom origins transformed by transaction extensions
- Feeless extrinsics
- XCM-gated authorization

Ring VRF, in context



People Chain



Another parachain

Ring roots propagate via XCM (`members-notifier` → `members-subscriber`)

you saw on Bulletin:

- Custom origins transformed by transaction extensions
- Feeless extrinsics
- XCM-gated authorization

Result: a feeless extrinsic on any chain that

- verifies a ring proof
- checks the alias hasn't been used for that context, and acts.

Anonymous. Single-use-per-context. Cross-chain.

Ring VRF, in context



People Chain



Another parachain

Ring roots propagate via XCM (`members-notifier` → `members-subscriber`)

you saw on Bulletin:

- Custom origins transformed by transaction extensions
- Feeless extrinsics
- XCM-gated authorization

Result: a feeless extrinsic on any chain that

- verifies a ring proof
- checks the alias hasn't been used for that context, and acts.

Anonymous. Single-use-per-context. Cross-chain.

If you are a registered person, some actions on Polkadot become free

Close & Q&A

- **TrUAPI**: A sandboxed product
- **Bulletin Chain**: Block authors who lose the data cannot author the next block.
- **Statement Store**: An off-chain subsystem, with offchain workers as the bridge
- **Ring VRF**: One primitive, reused many times

What we skipped

- pallet-revive internals: Solidity → PolkaVM, EVM compatibility details
- dotNS specifics beyond "Solidity contract on Asset Hub, resolves .dot names to CIDs."
- Host API specifics beyond the nine domains: SCALE encoding, codec negotiation, subscription lifecycle.
- HOP and P2P file transfer: Bulletin-adjacent
- Bulletin's data-renewal design: Active RFC
- The full Individuality ecosystem: DIMs, proof-of-ink, score / game pallets, coinage, resources
- Concrete sponsored-access designs. Being designed right now; I'll share when it's public.